BucketServe: Bucket-Based Dynamic Batching for Smart and Efficient LLM Inference Serving

Wanyi Zheng[1,2], Minxian Xu[2,*], Shengye Song[1,2], Kejiang Ye[2]
[1]Southern University of Science and Technology
[2]Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences
Email: {wy.zheng, mx.xu, sy.song, kj.ye}@siat.ac.cn

arXiv:2507.17120v1 [cs.DC] 23 Jul 2025

***Abstract*—Large language models (LLMs) have become increasingly popular in various areas, traditional business gradually shifting from rule-based systems to LLM-based solutions. However, the inference of LLMs is resource-intensive or latency-sensitive, posing significant challenges for serving systems. Existing LLM serving systems often use static or continuous batching strategies, which can lead to inefficient GPU memory utilization and increased latency, especially under heterogeneous workloads. These methods may also struggle to adapt to dynamic workload fluctuations, resulting in suboptimal throughput and potential service level objective (SLO) violations. In this paper, we introduce BucketServe, a bucket-based dynamic batching framework designed to optimize LLM inference performance. By grouping requests into size-homogeneous buckets based on sequence length, BucketServe minimizes padding overhead and optimizes GPU memory usage through real-time batch size adjustments preventing out-of-memory (OOM) errors. It introduces adaptive bucket splitting/merging and priority-aware scheduling to mitigate resource fragmentation and ensure SLO compliance. Experiment shows that BucketServe significantly outperforms UELLM in throughput, achieving up to 3.58× improvement. It can also handle 1.93× more request load under the SLO attainment of 80% compared with DistServe and demonstrates 1.975× higher system load capacity compared to the UELLM.**

***Index Terms*—LLM Serving, Disaggregated Architecture, Requests Orchestration, Resource Management.**

I. Introduction

In the past few years, with the advancement of the artificial intelligence (AI), large language models, such as GPT-4 [1], [2], Deepseek-R1 [3], and LLaMA [4], based on Transformer [5] architecture and its variants (e.g., Encoder-Only and Decoder-Only) have demonstrated remarkable capabilities in natural language processing (NLP). Benefiting from the attention mechanism inherent in the Transformer architecture, LLMs can capture long-range dependencies and contextual information, enabling them to perform a wide range of tasks, such as text generation, translation, summarization, multi-turn dialogue and delivering precise automated question-answering services [6]–[8]. Thereby expanding the practical applications of large-scale models. Leveraging their massive parameter scales and deep learning capabilities, these models effectively capture the complexity and semantic structures of human language, marking a significant step toward achieving general AI and human capability.

*M. Xu is the corresponding author.

However, these advancements in LLMs have also driven a growing demand for high-performance inference systems capable of meeting the computational and memory requirements of increasingly LLMs [9]–[15]. Unlike traditional online services that rely on lightweight models or database queries [9], LLM inference typically demands substantial GPU resources and storage capacity in AI infrastructure (e.g. cloud computing data centers). In particular, real-time services such as chatbots [16], virtual assistants, and interactive AI systems [9] require strict adherence to SLOs, where low latency and fast response times are essential to user satisfaction. Performance degradation for LLM inference services deployed on cloud can lead to poor user experience, reduced engagement, and potential business impact [17] [18]. On the other hand, there are applications with more relaxed latency constraints, where throughput and resource utilization are the primary concerns. In these cases, the focus should shift toward maximizing system efficiency and minimizing cost per request.

Therefore, it is crucial to design flexible and scalable LLM inference systems that can adapt to diverse application requirements [12]. For time-sensitive tasks, optimization efforts should prioritize reducing end-to-end latency and ensuring SLO compliance. For less latency-critical workloads, the emphasis should be on improving throughput and optimizing resource allocation [19]. To address this challenge, researchers and engineers have explored various techniques to enhance the end-to-end performance of LLM inference, including dynamic batching [20], and efficient scheduling strategies [19]. Among them, recent research has shown increasing interest in disaggregated architectures [9], [21], [22], such as DistServe [9], which decouples the prefill and decoding phases to enable specialized optimizations for each stage. Building upon such architectural innovations, further improvements, especially in request scheduling, remain possible and highly valuable. Empirical studies from major cloud providers demonstrate that intelligent scheduling can significantly improve both throughput and resource efficiency without compromising service quality [19]. This makes advanced scheduling strategies an essential component in next-generation LLM serving systems, particularly under high concurrency and heterogeneous workload conditions.

In this paper, we propose BucketServe, a bucket-based dynamic batching framework for efficient LLM inference, designed to achieve both high throughput and low latency

under concurrent workloads. Built upon vLLM [10], BucketServe extends its capabilities to support request scheduling in a disaggregated serving architecture. The core idea of BucketServe is to group incoming requests into buckets based on their sequence lengths, enabling fine-grained and adaptive batching decisions. This approach allows the system to prioritize latency-sensitive requests while maximizing overall throughput. Experimental results demonstrate that BucketServe significantly improves system efficiency and scalability compared to baselines while introducing negligible overhead. Our contributions can be summarized as follows:

- **Mitigate resource inefficiency in static batching strategies:** We address the critical issue of resource waste caused by naive batching approaches through request orchestration based on input characteristics (e.g., sequence length and memory footprint).
- **Introduce a bucket-based dynamic batching strategy:** We propose a novel algorithm that dynamically adjusts the border of the buckets and the batch size according to real-time GPU memory constraints and workloads while prioritizing high-priority requests.
- **Demonstrate the superior performance of BucketServe:** We comprehensively evaluate BucketServe on representative LLM serving workloads, showing its effectiveness in improving throughput and handling high-concurrency scenarios. Our results indicate that BucketServe significantly reduces SLO violations under high heterogeneous workloads compared to existing systems.

II. Background and Motivation

As LLMs are increasingly integrated into applications such as text generation and conversational AI, LLM-based services have rapidly transitioned from research prototypes to large-scale production systems. The dominant deployment paradigm is the client-server architecture, where user requests are funneled through a gateway to backend inference servers responsible for executing generative AI tasks and returning results. These services encompass both real-time interactive scenarios and offline batch processing tasks.

Despite their versatility, LLM inference remains highly resource-intensive: a single inference with a large model can occupy several gigabytes of GPU memory, and the autoregressive generation process leads to considerable variability in response latency, varying from seconds to minutes. These factors pose significant challenges for maintaining SLOs under high concurrency.

A. Challenges in Disaggregated Architecture

Disaggregating LLM inference into prefill and decoding phases is illustrated in Fig. 1, where the LLM reads the entire prompt at once and computes the Key-Value (KV) Cache in the prefill phase. During decoding phase, the model generates new tokens incrementally, one token at a time, based on the existing context and the previously computed KV cache. It continuously updates the KV cache and passes the newly generated token back to the model. The Disaggregation enables

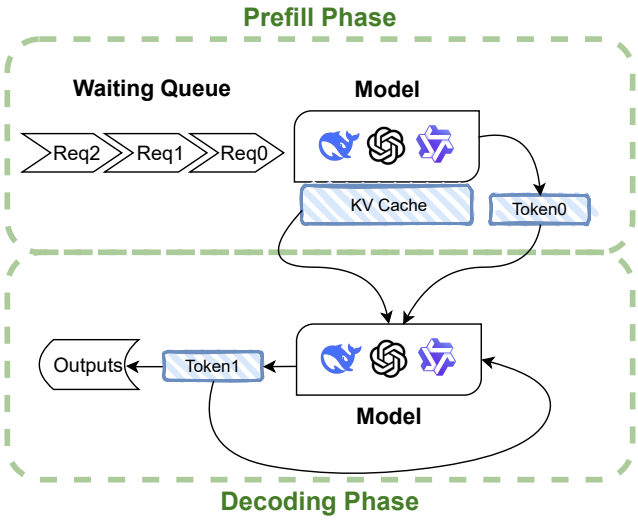


Fig. 1: Prefill and Decoding disaggregation architecture for LLM inference.

specialized optimizations for each stage, but also introduces several significant challenges that must be addressed to achieve efficient and reliable serving.

1) Resource Contention and Imbalance: The prefill phase is highly compute-intensive, requiring substantial GPU computation to process input prompts in parallel and construct the initial KV cache. In contrast, the decoding phase generates output tokens sequentially and is often bottlenecked by memory bandwidth due to frequent KV cache accesses. This heterogeneity can lead to resource imbalance, where compute resources may be under-utilized during decoding, while memory bandwidth becomes a limiting factor, resulting in suboptimal overall throughput.

2) Scheduling Complexity: Disaggregated architectures require two-stage scheduling: requests must be efficiently orchestrated for both prefill and decoding phases. This increases scheduling complexity, as requests may experience head-of-line blocking if long-sequence prefill jobs delay shorter ones, or if decoding jobs accumulate and saturate memory bandwidth. Moreover, scheduling strategies must account for request-specific attributes such as sequence length, priority, and latency sensitivity to avoid resource monopolization and ensure SLO compliance. Coordinating scheduling decisions across both phases is non-trivial, especially under heterogeneous and bursty workloads.

3) Batching Challenges: Batching is essential for maximizing GPU utilization, but in disaggregated architectures, the optimal batching strategy may differ between prefill and decoding. Grouping requests with diverse sequence lengths in a batch leads to excessive padding, wasting GPU memory and compute. The prefill phase benefits from large, homogeneous batches to maximize parallelism, while the decoding phase, due to its sequential nature and variable output lengths, may require smaller or more dynamic batches. Designing batching

strategies that adapt to the requirements of both phases, minimize padding, and avoid fragmentation is a key challenge.

4) Coordination and Data Transfer Overhead: The separation of prefill and decoding phases necessitates transferring large KV cache between them, often across GPUs or nodes. This introduces additional coordination and communication overhead, which can degrade latency and throughput if not carefully managed.

For the first and fourth challenges, which are highly dependent on hardware characteristics, we adopt the best configurations recommended by prior work [9]. The remaining challenges require advanced scheduling and batching mechanisms that are aware of both request characteristics and system resource constraints.

B. Opportunities

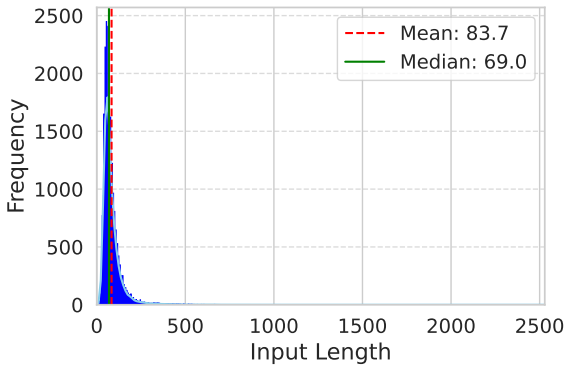
As shown in Figs. 2 and 3, here we use Stanford Alpaca [23] and LongBench [24] datasets to conduct the case study. Our analysis of request characteristics and phase-specific resource utilization reveals two critical opportunities for optimization:

Dynamic Batching with Prefill Bucketing. The heterogeneous sequence length distribution (Fig. 2) shows that the input lengths exhibit a heterogeneous distribution, with Alpaca sequences averaging 83 tokens and LongBench sequences showing a long-tail pattern (median 41,417 tokens). For LongBench's ultra-long sequences, we truncate them to the model and the dominance of long sequences in execution time (Fig. 3a) motivate a two-stage batching strategy. For the prefill phase, we adopt bucketing-based batching: requests are grouped into size-homogeneous buckets (e.g., [0–256 tokens], [256–1024 tokens]) to minimize padding overhead. This allows direct batch formation from buckets with predictable memory requirements, ensuring efficient GPU utilization for static input sequences. In contrast, for the decoding phase that have output lengths, we apply continuous batching [20] to dynamically incorporate newly arriving tokens without waiting for full batches. This combination leverages the static nature of prefill inputs while adapting to the dynamic decoding process.

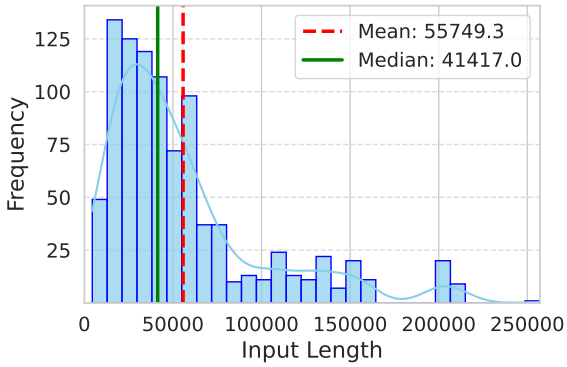
Bucket-Aware Scheduling. The low GPU utilization observed in mixed-batch (containing samples from two datasets) decoding (Fig. 3b) highlights the need for scheduling strategies that align with workload characteristics. By first partitioning requests into buckets with minimal overhead, then applies phase-aware dispatching tailored to specific performance goals:

In scenarios where requests per second (RPS) is the priority, we consider applying shortest-job-first (SJF) within buckets to minimize queuing latency and maximize throughput. By prioritizing shorter requests, this strategy ensures timely responses for latency-sensitive tasks, even at the cost of reduced GPU memory utilization for long sequences.

For workloads prioritizing token-per-second throughput, we can use longest-job-first (LJF) within buckets to group long sequences. This maximizes GPU utilization during decoding by leveraging parallelized attention operations and reducing



(a) Distribution of Alpaca.



(b) Distribution of LongBench.

Fig. 2: Distribution of LLM Requests.

padding overhead, albeit at the expense of increased latency for individual requests.

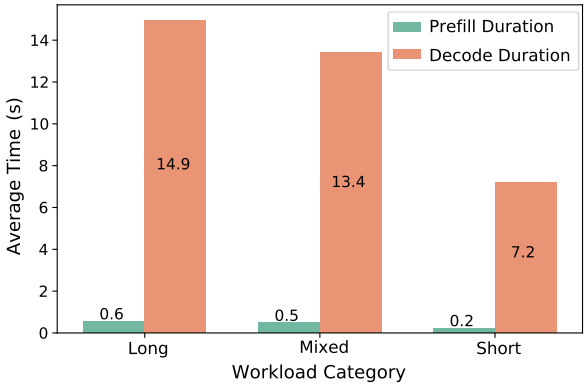
We address these issues by proposing adaptive bucketing and dynamic batching strategies proposed above that jointly optimize resource utilization, scheduling, and memory efficiency across both phases of disaggregated LLM inference. The implementation details are presented in Section IV.

III. System Overview

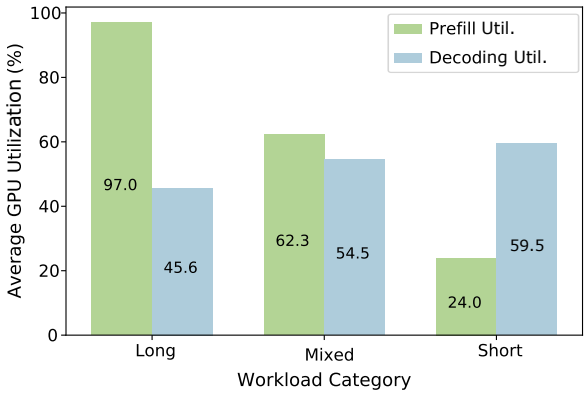
As illustrated in Fig. 4, BucketServe adopts a three-tier architecture, decoupling application workloads from hardware resources via a middleware layer. The top-tier application layer supports diverse LLM services, including both online and offline tasks. When incoming requests arrive at the gateway, they are routed to the appropriate services based on their type and priority. The middleware layer then performs adaptive request bucketing, grouping requests into buckets according to their sequence length and task category.

The system employs a dynamic bucketing strategy to optimize resource utilization across varying workloads:

Low-Load Scenarios: In situations where the RPS is low and all requests can be processed in a single batch, the system consolidates all requests into a single bucket to minimize scheduling overhead.



(a) Average Batch Execution Cost.



(b) Average GPU Utilization.

Fig. 3: Performance of batch execution and GPU utilization across different workload types. Long refers to those exceeding 1024 tokens sampled from LongBench, Short denotes those under 256 tokens from Alpaca, and Mixed contains sequences from both datasets following a long-tail distribution pattern.

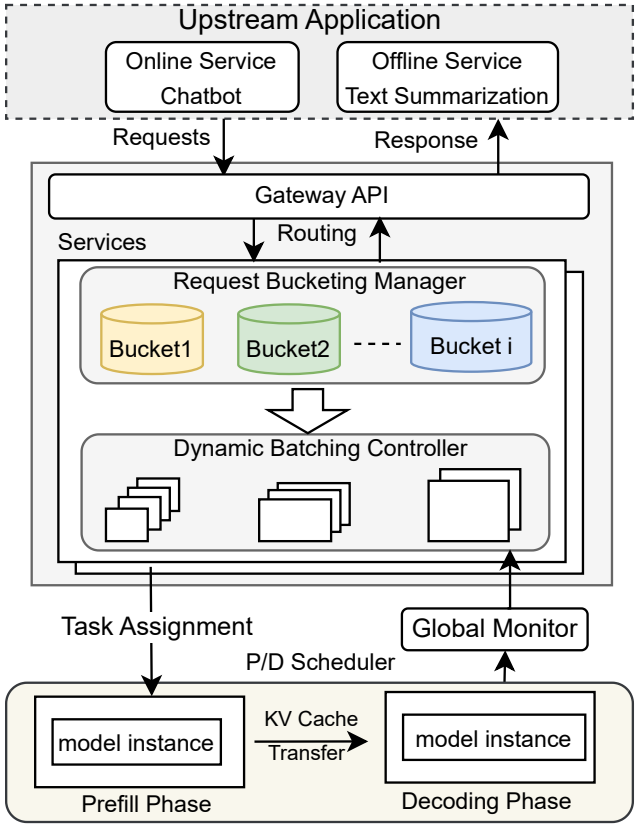


Fig. 4: The Architecture of BucketServe.

High-Load Scenarios: If the number of requests in a bucket exceeds a threshold and exhibits uneven distribution (e.g., a mix of short and long sequences), the system dynamically adjusts the bucket count and boundaries to reduce padding overhead and improve batch efficiency. This adaptive mechanism ensures optimal performance across diverse workloads and is discussed in detail in Section IV.

Request Bucketing Manager is responsible for grouping incoming requests into buckets based on their sequence lengths and task categories. It dynamically adjusts the number of buckets and their boundaries to minimize padding overhead and ensure efficient batching. In low-load scenarios, it consolidates all requests into a single bucket to reduce scheduling overhead. In high-load scenarios, it splits buckets when the number of requests exceeds a threshold or when the distribution of sequence lengths becomes uneven. This adaptive approach helps maintain optimal resource utilization and throughput across diverse workloads.

Dynamic Batching Controller takes requests from the buckets and groups them into batches for processing. It dynamically computes the optimal batch size based on current GPU memory constraints and bucket boundaries. By adjusting the batch size according to available resources, the controller prevents OOM errors while maximizing throughput. Additionally, the controller prioritizes requests that have been waiting the longest, ensuring they are processed quickly and efficiently.

P/D Scheduler manages the transition of requests between the prefill phase and the decoding phase. Once the requests are batched, they are assigned to the waiting queue of the prefill phase. A First-Come-First-Served (FCFS) scheduling strategy is adopted by the workers to process the requests in order of their arrival. This ensures fairness and simplicity in execution order. After the prefill phase completes, the corresponding KV cache is transferred to the decoding phase via NVLink, enabling high-bandwidth, low-latency communication between GPUs. In the decoding phase, continuous batching is applied, as the output length during token generation is typically variable and hard to predict.

Global Monitor continuously collects and aggregates system-wide metrics, including GPU memory usage, queue lengths, request arrival rates, average sequence length, and batch processing latency. These metrics provide a real-time view of resource availability and workload characteristics, enabling the system to make data-driven decisions.

In particular, the Global Monitor feeds critical information to the Dynamic Batching Controller and P/D Scheduler. For

example, it informs the batching controller about current memory pressure and expected future demand, allowing it to adjust batch sizes dynamically without risking OOM errors. It also provides the scheduler with queue occupancy and waiting time statistics, supporting fair and efficient scheduling policies.

IV. Adaptive Bucketing and Dynamic Batching with Memory Safety

In this section, we present the design and implementation of our strategy. For offline tasks with relaxed latency constraints, requests are grouped into buckets based on sequence length to minimize padding overhead. For online tasks requiring low latency, the number of buckets and their boundaries are dynamically optimized in real-time. The batching module computes optimal batch sizes using the following equations to maximize throughput while preventing OOM errors.

The KV cache memory footprint for a batch is calculated as:

$$\begin{aligned}\text{Memory}_{\text{KV cache}} = &2 \times L \times H \times D \times \\ &S_{\max} \times B \times N,\end{aligned} \tag{1}$$

where L is the number of layers, H is the number of attention heads, D is the dimension per head, $S_{\max}$ is the maximum sequence length in the batch, B is the bytes per element (e.g., 2 for FP16), and N is the batch size.

To quantify the inefficiency introduced by padding, we define the **wasted memory ratio** as:

$$\text{Waste}_{\text{Ratio}} = \frac{S_{\max} - S_{\text{avg}}}{S_{\max}}, \tag{2}$$

where S_{avg} is the average sequence length in the batch. Bucketing reduces $S_{\max} - S_{\text{avg}}$, thereby minimizing memory waste.

To further analyze the expected memory waste across all buckets, we define the **expected waste rate** as:

$$\mathbb{E}[\text{Waste}] = \sum_{b=1}^{K} \int_{L_b}^{U_b} \left(1 - \frac{S}{U_b}\right) f(S) dS, \tag{3}$$

where $f(S)$ is the distribution of incoming request lengths of batch S, and $[L_b, U_b)$ denotes the range of the b-th bucket. This formulation provides a more comprehensive view of how different bucketing strategies affect overall padding overhead.

The **optimal bucket boundary** that minimizes $\mathbb{E}[\text{Waste}]$ is given by:

$$U_b^* = \frac{\int_{L_b}^{U_b} S f(S) dS}{\int_{L_b}^{U_b} f(S) dS}. \tag{4}$$

This indicates that the upper bound of each bucket should be set to the **conditional expectation** of sequence lengths within that bucket. Our adaptive algorithm approximates this condition dynamically during runtime.

Although the optimal bucket boundary that minimizes **E**[Waste] is theoretically defined as the conditional expectation of sequence lengths within a bucket, it is computationally expensive to calculate in practice. Moreover, since request length distributions can change over time, maintaining such boundaries dynamically introduces significant overhead and algorithmic complexity. To address this challenge, we adopt a simple but efficient approach based on interval bisection, which approximates the optimal boundary while keeping the system lightweight and responsive.

Algorithm 1 Adaptive Bucketing Mechanism

Require: Buckets B, maximum length $L_{\max}$
Require: Minimum split size $m = N_{\max}$, threshold $\theta = 0.5$
Initialize $B \leftarrow \{[0, L_{\max})\}$
for each request $r = (S, t, \texttt{type})$ **do**
  for each bucket $b \in B$ **do**
    if $b_{\text{low}} \le S < b_{\text{up}}$ **then**
      Add r to $b.\texttt{requests}$
      break
    end if
  end for
end for

function AdjustBuckets
  $\texttt{total} \leftarrow \sum_b |b.\texttt{requests}|$
  if $\texttt{total} < N_{\max}$ **then**
    $B \leftarrow \{[0, L_{\max})\}$
  else
    $\texttt{split_list} \leftarrow \emptyset$
    for each bucket $b \in B$ **do**
      $m_b \leftarrow (b_{\text{low}} + b_{\text{up}})/2$
      $C_s \leftarrow |\{r \in b.\texttt{requests} \mid r.S < m_b\}|$
      if $C_s/|b.\texttt{requests}| > \theta$ **and** $|b.\texttt{requests}| > m$ **then**
        $\texttt{split_list} \leftarrow \texttt{split_list} \cup \{b\}$
      end if
    end for
    for each $b \in \texttt{split_list}$ **do**
      mid $\leftarrow (b_{\text{low}} + b_{\text{up}})/2$
      Create $b_L \leftarrow [b_{\text{low}}, \text{mid})$
      Create $b_R \leftarrow [\text{mid}, b_{\text{up}})$
      Partition $b.\texttt{requests}$ into b_L, b_R by S
      $B \leftarrow B \cup \{b_L, b_R\} \setminus \{b\}$
    end for
  end if
end function

To ensure memory safety, the scheduler reserves 10% of GPU memory for system overheads. The **safe available memory** is computed as:

$$M_{\text{safe}} = 0.9 \times M_{\text{remain}}, \tag{5}$$

where M_{remain} is the remaining memory after model allocations.

The **maximum safe batch size** is determined as:

$$N_{\max} = \max\left\{ N \in \mathbb{N} \;\middle|\; \sum_{i=1}^{N} S_i \le \frac{M_{\text{safe}}}{2LHDB} \right\}, \tag{6}$$

which ensures that the KV cache remains within safe limits.

Algorithm 1 implements an adaptive bucketing mechanism to manage request scheduling and resource allocation. Initially, the system starts with a single bucket covering the full sequence length range $[0, L_{\max})$. Requests are assigned to the appropriate bucket based on their sequence length S (lines 2-6), where each incoming request is matched against the current set of buckets.

When the total number of requests across all buckets exceeds $N_{\max}$ (computed via Eq. (6)), the algorithm triggers a splitting strategy. Specifically, for each bucket b, if over 50% of its requests have sequence lengths below the midpoint $m_b = (b_{\text{low}} + b_{\text{up}})/2$ and the bucket contains more than $N_{\max}$ requests, it is split into two sub-buckets. This midpoint-based decision (lines 14-19), serves as an approximation to the optimal boundary defined in Eq. (4). Splitting helps reduce padding by grouping similar-length sequences together. Then it conducts a partitioning of requests into the new buckets based on their sequence lengths (lines 23-29). This process continues until all buckets are split depending on the current workload.

Conversely, when the total number of requests drops below $N_{\max}$, all buckets merge back into a single one to minimize scheduling overhead (lines 11-13). After bucketing, offline tasks use SJF or LJF scheduling within buckets to optimize throughput, while online tasks prioritize buckets based on earliest request arrival time to meet SLOs. The algorithm balances efficiency and fairness through dynamic adjustments, leveraging thresholds like $\theta = 0.5$ for splitting decisions and $N_{\max}$ for batch size constraints.

The potential performance improvements include optimizing bucket search with data structures such as binary trees and refining splitting criteria using distribution-aware methods.

Algorithm Complexity Analysis. The adaptive bucketing algorithm consists of two main components. The first is assigning requests to buckets: for each of the n requests, the algorithm traverses all k buckets to find the appropriate one, resulting in a time complexity of $O(n \cdot k)$. The second component is the `AdjustBuckets` function. Calculating the total number of requests requires $O(k)$ time. For each bucket, the algorithm performs constant-time computations and checks, also totaling $O(k)$. Splitting buckets and partitioning requests can be done in $O(k)$ time overall. Therefore, the time complexity of the adjustment step is $O(k)$. In summary, the overall time complexity of the adaptive bucketing mechanism is $O(n \cdot k + 4k)$.

V. Performance Evaluations

In this section, we evaluate the performance of BucketServe compared with two baseline systems. Because BucketServe is originally designed for offline tasks which have high throughput requirement, we adapted it to online tasks, we found that the performance of BucketServe outperforms baseline systems in high concurrency scenarios.

A. Experiments Setup

Testbed. We evaluated BucketServe on a single-node cluster equipped with 4 NVIDIA A100 40GB GPUs interconnected via NVLink for high-bandwidth GPU-to-GPU communication. The node also features 64 CPU cores and 1TB of high-speed SSD storage. Our serving framework is built on the vLLM backend, which supports continuous batching and paged attention for high-performance inference.

Models and Datasets. We selected the LLaMA-2 [25] and OPT [26] series as representative foundation models due to their standardized architectures and widespread adoption in academic research. To evaluate performance across diverse task profiles, we used two benchmarking datasets: Stanford Alpaca [23], characterized by short sequence lengths, and LongBench [24], designed for long-text summarization scenarios. The Mixed configuration refers to a hybrid of these two datasets. This setup enables a comprehensive evaluation spanning both short and long sequence processing paradigms.

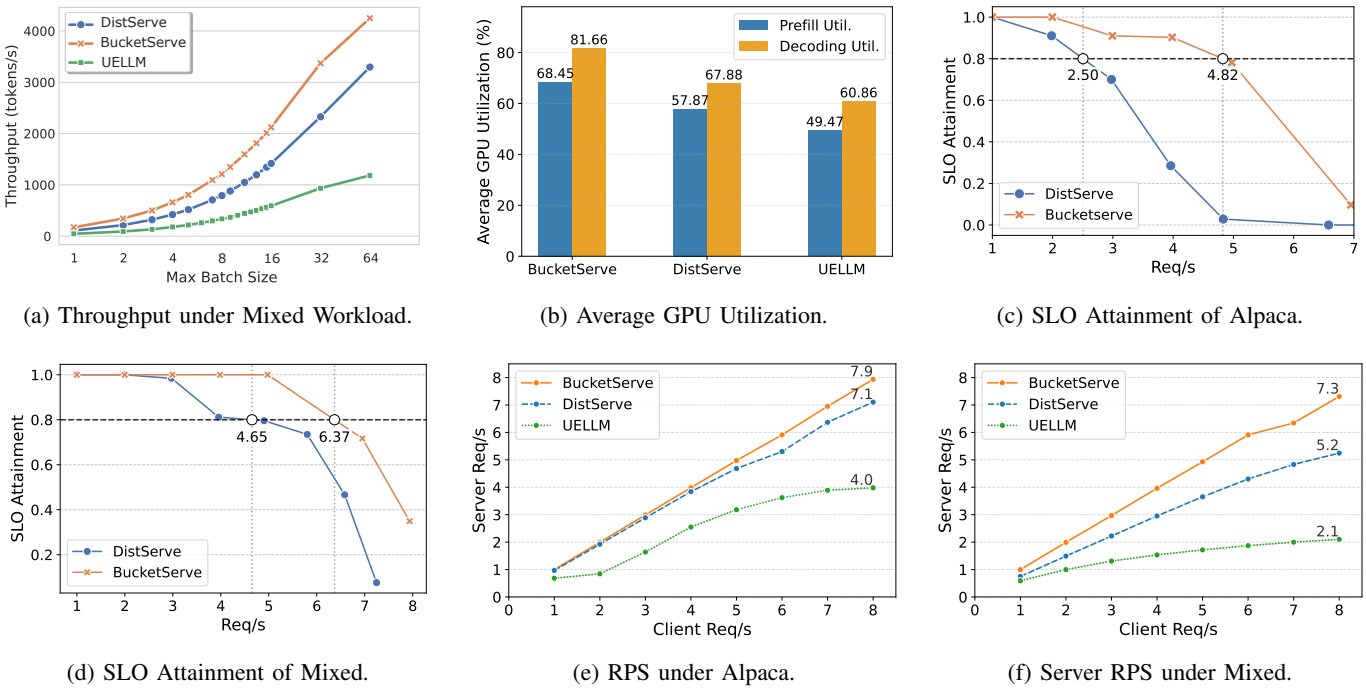
Metrics. BucketServe's performance evaluation employs two distinct metric categories tailored to task types. For offline workloads, we prioritize throughput (measured in tokens per second) and pay attention to resource utilization to quantify system efficiency in handling large-scale request volumes. In online scenarios, we adopt SLOs attainment rate and service load capacity as the primary metrics.

Baselines. We compare BucketServe with two state-of-the-art baseline systems including both aggregated and disaggregated architectures:

- **UELLM** [12]: A unified LLM serving framework that integrates resource profiling, batch scheduling, and deployment optimization. It employs a fine-tuned LLM to predict resource demands, batches queries based on predicted profiles, and deploys models considering hardware topology. However, UELLM still couples prefill/decoding phases and lacks dynamic adaptation to workload fluctuations, leading to suboptimal latency-SLO trade-offs under heterogeneous traffic patterns.
- **DistServe** [9]: A disaggregated serving system that decouples prefill and decoding computation to eliminate phase interference. It co-optimizes resource allocation for each phase and minimizes communication overhead via bandwidth aware placement. While under high concurrency, DistServe lacks specialized process leading to suboptimal performance in heterogeneous workloads.

B. Experiments Analysis

We first evaluate the performance of BucketServe in offline tasks with those two baselines. To study the throughput and average resource utilization, we conducted experiments using Llama2-13B with varying batch sizes and sequence lengths. We randomly sampled requests from the Alpaca and LongBench datasets to simulate real-world workloads. The results are shown in Fig. 5a and Fig. 5b. The max batch size represents the largest number of requests the system can actually handle in one batch under the current setup, reflecting its realizable capacity. From this figure, we can see that BucketServe achieves the highest throughput and average GPU utilization especially under more requests. Specifically, BucketServe outperforms UELLM by 3.58× and DistServe



(a) Throughput under Mixed Workload.

(b) Average GPU Utilization.

(c) SLO Attainment of Alpaca.

(d) SLO Attainment of Mixed.

(e) RPS under Alpaca.

(f) Server RPS under Mixed.

Fig. 5: End-to-End performance evaluation across offline and online dimensions.

1.31× in throughput under high workloads, While dynamic batching improve average GPU utilization to 81.66%. This demonstrates the effectiveness of our adaptive bucketing and dynamic batching strategy in optimizing resource utilization and throughput.

For online tasks, we conducted experiments under varying client RPS and SLO requirements across different datasets. As shown in Fig. 5c and Fig. 5d, we compared BucketServe with DistServe using two datasets. We measured the server RPS and analyzed its relationship with SLO attainment.

As the server RPS increases, the SLO attainment of both systems decreases. At an SLO attainment level of 80%, BucketServe achieves 1.37× and 1.93× higher server RPS than DistServe on the Alpaca and Mixed datasets, respectively. This indicates that our system performs better under high workloads and for heterogeneous length requests.

We also studied the relationship between the client request sending rate and the server processing rate, as it reflects the system's capability to handle high concurrency. From Fig. 5e and Fig. 5f, we observe that, under both datasets, BucketServe's server RPS closely follows the ideal y = x line, indicating excellent scalability and efficiency.

On the Alpaca dataset, DistServe shows little performance degradation, but BucketServe still outperforms UELLM by 1.975× in terms of server RPS. On the Mixed dataset, BucketServe exhibits almost no performance degradation and achieves 1.4× and 3.47× higher server RPS than DistServe and UELLM, respectively. These results demonstrate BucketServe's superior effectiveness, particularly under high workloads and for long-sequence processing.

Fig. 6 presents a breakdown of the end-to-end execution duration for BucketServe and the associated bucketing overhead. As shown in Fig. 6a, the decoding phase constitutes the majority of the execution time, accounting for approximately 90% of the total under typical workloads. When the RPS is set to 32, some requests are queued waiting for the prefill stage, resulting in an increased average prefill duration.

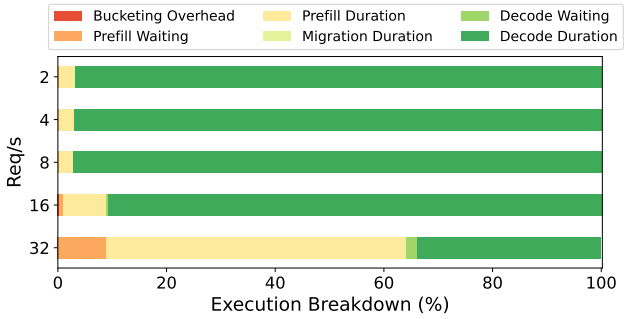
The red bar representing bucketing overhead is barely visible in the figure, indicating that the cost of bucketing is relatively low. As a result, our system introduces negligible overhead from bucketing and dynamic batching—less than 1% of the total execution time. This highlights the efficiency of our adaptive bucketing and dynamic batching strategy in optimizing resource utilization and improving throughput.

Fig. 6b further illustrates the bucketing overhead. It shows that as the number of buckets increases, the algorithmic overhead remains stable, demonstrating its scalability and computational efficiency.

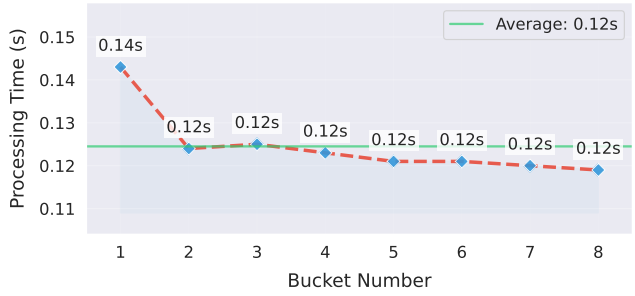
To summarize, by introducing bucketing and dynamic batching, BucketServe effectively addresses the challenges of resource under-utilization and scheduling complexity in disaggregated LLM inference architectures, while introducing negligible overhead and maintaining performance comparable to prior approaches.

VI. Related Work

Recent advancements in LLM inference optimization focus on enhancing end-to-end performance through diverse approaches [9]–[14], [19], [21], [22], [27], [28], which can be broadly categorized into three domains: prefill-decoding disaggregation architectures, attention mechanism optimizations, and dynamic batching techniques.



(a) Execution Duration Breakdown.



(b) Distribution of Mixed.

Fig. 6: End-to-End Latency Breakdown and Bucketing Overhead Analysis.

Prefill-Decoding Disaggregation. This optimization technique aims to decouple the distinct computational phases of LLM inference—Prefill and Decoding—to improve resource allocation. DistServe [9] pioneers this approach introducing an optimal configuration algorithm to dynamically allocate GPU resources and model parallelism strategies. In its design, to prioritize Time-to-First-Token (TTFT), waiting Prefill requests can preempt Decoding resources. While this reduces TTFT, it increases tail latency for Time-Between-Tokens (TBT) by up to 30% due to frequent context switching. SplitWise [21] introduces a Prompt Phase and Token-Generation Phase, decomposing tasks into subtasks and mapping them to heterogeneous hardware via layer-wise KV cache transfer. This approach achieves 40% higher resource utilization through task parallelism but incurs overhead in short-sequence workloads, increasing end-to-end latency. TetriInfer [28] partitions prompts into fixed-size chunks and employs a two-level scheduler to optimize batching, reducing average TTFT by 97% and JCT by 47% while cutting resource usage by 38%. However, its chunked computation introduces communication overhead in high-throughput, long-sequence scenarios. Mooncake [29] decouples KV cache storage from computation, leveraging Chunked Pipeline Parallelism (a variant of TeraPipe [30]) and prediction-guided early rejection to improve request throughput by 75% under high load. However, inaccuracies in prediction models (>15% error rates) lead to false rejections, harming throughput. LoongServe [31] introduces Elastic Sequence Parallelism, dynamically adjusting model parallelism degrees and optimizing KV cache migration to achieve higher throughput, while consistency maintenance for long sequences (1000+ tokens) adds significant overhead (e.g. 18% computation). BucketServe also applies this architecture while using bucket-based scheduling to enhance system performance.

Attention Mechanism. This type of technology focuses on reducing memory and computational bottlenecks in the attention module. vLLM's PagedAttention [10] organizes KV cache into fixed-size blocks (similar to OS virtual memory), reducing fragmentation from 35% to 5% and boosting throughput by 2.4×. FlashAttention [27] fuses attention operations into a single kernel, accelerating computation by 1.8× on A100 GPUs through tiling and SRAM-based intermediate storage. FlashAttention-2 [13] further improves parallelization and supports advanced features like multi-query attention. Grouped Query Attention [32] balances efficiency and quality by grouping attention heads, enabling scalable context windows and batch sizes while retaining near-MHA performance. However, these work do not focus on requests scheduling.

Dynamic Batching. It aims to maximize GPU utilization by adapting to variable-length sequences while currently limited work consider batching dynamically. Orca [20] employs iteration-level scheduling, where batch sizes are determined per forward-backward pass. By replacing completed sequences immediately, it achieves higher GPU utilization than static batching, however, it only considers simple P/D disaggregation. FastServe [33] uses preemptive scheduling with a skip-join Multi-Level Feedback Queue to minimize JCT. Leveraging the semi-stateless nature of LLM inference, it dynamically prioritizes jobs based on input length, skipping lower-priority queues to reduce latency. FastServe also integrates efficient GPU memory management to handle variable workloads while the batch algorithm is not adaptive to dynamic workloads.

VII. Conclusions and Future Work

In this paper, we present BucketServe, a bucket-based dynamic batching framework designed to address the challenges of efficient LLM inference serving under concurrent and heterogeneous workloads. BucketServe introduces an adaptive bucketing mechanism that groups requests by sequence length, thereby reducing padding overhead and improving GPU memory utilization. The system further employs a dynamic batching controller that adjusts batch sizes in real time based on current GPU memory constraints, ensuring memory safety and maximizing throughput. Our comprehensive evaluations demonstrate that BucketServe achieves substantial improvements over existing baselines, including up to 3.58× higher throughput and nearly 2× greater system load capacity, while maintaining SLO attainment and incurring minimal overhead. These results validate the effectiveness of combining adaptive bucketing with dynamic batching for scalable, high-performance LLM serving. As for future work, we would like to explore multi-level load balancing strategies that can further enhance the system's performance under varying workloads on multi-nodes clusters.

ACKNOWLEDGMENTS

This work is supported by Guangdong Basic and Applied Basic Research Foundation (No. 2024A1515010251, 2023B1515130002), Guangdong Special Support Plan (No. 2021TQ06X990), Key Research and Development and Technology Transfer Program of Inner Mongolia Autonomous Region (2025YFHH0110), Shenzhen Basic Research Program under grants JCYJ20220818101610023 and JCYJ20240809180935001.

REFERENCES

[1] OpenAI, J. Achiam, S. Adler, S. Agarwal, and et.al, "Gpt-4 technical report," 2024. [Online]. Available: https://arxiv.org/abs/2303.08774

[2] Y. Dang, Y. He, M. Xu, and K. Ye, "Resource management for gpt-based model deployed on clouds: Challenges, solutions, and future directions," in *Algorithms and Architectures for Parallel Processing: 24th International Conference, ICA3PP 2024, Macau, China, October 29–31, 2024, Proceedings, Part II*, 2025, p. 95–105.

[3] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, and et.al, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[4] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, and et.al, "The llama 3 herd of models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[6] J. Li, T. Tang, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "Pre-trained language models for text generation: A survey," *ACM Computing Surveys*, vol. 56, no. 9, pp. 1–39, 2024.

[7] G. Adams, A. R. Fabbri, F. Ladhak, E. Lehman, and N. Elhadad, "From sparse to dense: Gpt-4 summarization with chain of density prompting," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing.*, vol. 2023, no. 4th New Frontier Summarization Workshop, 2023, p. 68.

[8] Z. Yi, J. Ouyang, Y. Liu, T. Liao, Z. Xu, and Y. Shen, "A survey on recent advances in llm-based multi-turn dialogue systems," *arXiv preprint arXiv:2402.18013*, 2024.

[9] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: disaggregating prefill and decoding for goodput-optimized large language model serving," in *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'24. USA: USENIX Association, 2024.

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[11] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," in *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'24. USA: USENIX Association, 2024.

[12] Y. He, M. Xu, J. Wu, W. Zheng, K. Ye, and C. Xu, "Uellm: A unified and efficient approach for large language model inference serving," in *Service-Oriented Computing: 22nd International Conference, ICSOC 2024, Tunis, Tunisia, December 3–6, 2024, Proceedings, Part I*, 2024, p. 218–235.

[13] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[14] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley *et al.*, "Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.

[15] B. Li, Y. Jiang, V. Gadepally, and D. Tiwari, "LLM inference serving: Survey of recent advances and opportunities," in *IEEE High Performance Extreme Computing Conference, HPEC 2024, Wakefield, MA, USA, September 23-27, 2024*. IEEE, 2024, pp. 1–8.

[16] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," 2023.

[17] H. Bai, M. Xu, K. Ye, R. Buyya, and C. Xu, "Drpc: Distributed reinforcement learning approach for scalable resource provisioning in container-based clusters," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3473–3484, 2024.

[18] L. Wen, M. Xu, S. S. Gill, M. Hilman, S. N. Srirama, K. Ye, and C. Xu, "Statuscale: Status-aware and elastic scaling strategy for microservice applications," *ACM Trans. Auton. Adapt. Syst.*, vol. 20, no. 1, Mar. 2025.

[19] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.

[20] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 521–538.

[21] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *51st ACM/IEEE Annual International Symposium on Computer Architecture, ISCA 2024, Buenos Aires, Argentina, June 29 - July 3, 2024*. IEEE, 2024, pp. 118–132.

[22] C. Hu, H. Huang, J. Hu, J. Xu, X. Chen, T. Xie, C. Wang, S. Wang, Y. Bao, N. Sun, and Y. Shan, "Memserve: Context caching for disaggregated LLM serving with elastic memory pool," *CoRR*, vol. abs/2406.17565, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2406.17565

[23] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," https://github.com/tatsu-lab/stanford_alpaca, 2023.

[24] Y. Bai, X. Lv, J. Zhang, H. Lyu, Tang, and et.al, "LongBench: A bilingual, multitask benchmark for long context understanding." Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 3119–3137. [Online]. Available: https://aclanthology.org/2024.acl-long.172

[25] H. Touvron, L. Martin, K. Stone, P. Albert, and et.al, "Llama 2: Open foundation and fine-tuned chat models," 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[26] S. Zhang, S. Roller, N. Goyal, M. Artetxe, and et.al, "Opt: Open pre-trained transformer language models," 2022. [Online]. Available: https://arxiv.org/abs/2205.01068

[27] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[28] C. Hu, H. Huang, L. Xu, X. Chen, J. Xu, S. Chen, H. Feng, C. Wang, S. Wang, Y. Bao, N. Sun, and Y. Shan, "Inference without interference: Disaggregate LLM inference for mixed downstream workloads," *CoRR*, vol. abs/2401.11181, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2401.11181

[29] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot," in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. Santa Clara, CA: USENIX Association, Feb. 2025, pp. 155–170.

[30] Z. Li, S. Zhuang, S. Guo, D. Zhuo, H. Zhang, D. Song, and I. Stoica, "Terapipe: Token-level pipeline parallelism for training large-scale language models," in *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, vol. 139. PMLR, 2021, pp. 6543–6552.

[31] B. Wu, S. Liu, Y. Zhong, P. Sun, X. Liu, and X. Jin, "Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles, SOSP 2024, Austin, TX, USA, November 4-6, 2024*. ACM, 2024, pp. 640–654.

[32] N. Shazeer, "Fast transformer decoding: One write-head is all you need," *CoRR*, vol. abs/1911.02150, 2019. [Online]. Available: http://arxiv.org/abs/1911.02150

[33] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin, "Fast distributed inference serving for large language models," *CoRR*, vol. abs/2305.05920, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2305.05920